MCI project First Milestone Report

Team number : OS1B
Project Title: Augmented Security Knowledge Base

Milestone 1	Activities	Planned Outputs	Achieved Outputs
Restate the milestone from your Draft plan .	Restate the key activities from your draft plan.	Restate the planned outputs from your draft work plan.	Outline the actual outputs compared to what was projected (or type “same as planned”)
Product Delivery	Classifier Development	Build and train a classification model to categorize questions	same as planned
	Vectorized Dataset & Similarity Calculation	Build a Semantic Vector Database	same as planned
	Sentiment Analysis	Sentiment/style analysis result	same as planned
	LLM & Prompt	Integrate a lightweight LLM-based solution to generate summarized	same as planned
	Set up command line tool & Minimum Viable Product(MVP)	CLI interface to accept natural language security questions as input & Deploy the whole web application.	same as planned
Team reflection on progress	Provide some comments below regarding the completion of this milestone specifically around: 1. How is the project progressing? 2. Are there any differences between projected and actual outputs/outcomes?		
The project is progressing well. All key components—including backend development, frontend interface design, and model integration—have been implemented as planned. The frontend user interface is functional and aligned with the backend service. Both frontend and backend have been successfully deployed. No significant differences. All components, including the classifier, sentiment analyzer, vector database, and LLM prompt integration, were delivered as expected. The frontend was slightly refined based on user feedback, but overall outcomes align with the original plan.			

Team reflection on managing problems	Have you encountered any problems to date? If so, how have you managed them?

1.**SVM, LDA**

Three major challenges were encountered: SVM implementation and LDA topic modeling.

1.1 For SVM, the initial classification accuracy was 52%, which was relatively low. To improve performance, we adopted the all-MiniLM-L6-v2 model — a transformer-based model known for capturing semantic relationships effectively and performing well in classification tasks. Additionally, class weights were applied to handle class imbalance and improve robustness. With these enhancements, the accuracy increased to 69%.

1.2 In the LDA workflow, the primary issues included determining the optimal number of topics and selecting an appropriate preprocessing strategy. Preprocessing steps from a referenced research paper were followed, and the number of topics was selected using coherence scores as the evaluation metric. ChatGPT 4o was used to generate concise and descriptive topic names.

2. Evaluating Similarity Search Accuracy

There are two challenges encountered in the similarity check part. simulate input from user and the evaluation method based on return result

2.1 Create test data

One challenge we encountered was evaluating the effectiveness of different similarity metrics for retrieving relevant StackOverflow posts. To reduce manual effort, we leveraged an LLM for Ollama in the llama3 version to simulate user input by rewriting questions and mapping them to original post IDs. This allowed us to automate evaluation across multiple similarity strategies, identifying accuracy and speed trade-offs.

2.2 Evaluate based on result

To evaluate the effectiveness of different similarity computation methods, we focused on two key aspects:

Retrieval Accuracy:

Given that our system is designed to return multiple relevant database entries per user query, we assessed whether the rewritten question could successfully retrieve its corresponding original post from the database. This metric reflects the model's ability to preserve semantic meaning across paraphrasing and similarity search.

Retrieval Speed:

Since response time is critical for user experience, we also measured the average time each similarity method took to return top-k (which is defaultly set to 3) results. This allowed us to balance retrieval quality with computational efficiency.

3. Frontend and Backend technique design

3.1 How to design Frontend techniques?

We chose React and Tailwind CSS[1] for the Frontend because:

- Component-Based Architecture.
- Styling Flexibility with Tailwind CSS
- Responsive Design

3.2 How to design backend techniques?

We chose FastAPI and LangChain[3] for the Backend because:

- Web Framework: FastAPI is used as the lightweight Python web server to handle API requests. Since many AI-related tools and libraries are built in Python, using FastAPI helps keep the technology stack simple and consistent.
- LLM Integration: Used LangChain to construct prompt pipelines and interact with a locally deployed LLM (via Ollama).

4. How to choose an LLM model?

4.1 Choose which LLM model as our local LLM?

we choose **llama3** considering the following reasons:

1. Llama3 Top-ranked in Instruction-Following Evaluation (IFEval) in hugging face leaderboard[2];
2. Fully open-source and commercially permissive license Compared with ChatGPT;
3. Optimized model size for local deployment(8B version of LLaMA 3 requires approximately 4GB of VRAM);
4. Strong community support and ecosystem tools(such as LangChain)[6];

5. How to design User Interface and the structure of response?

5.1 Choose the visual design elements for User Interface

- Select a color scheme: Choose colors that are visually appealing, consistent with the brand, and aid in readability and usability.
- Pick typography: Select fonts that are easy to read and appropriate for the context. Use font sizes and weights to create a visual hierarchy and distinguish between different types of information.
- Use icons and images: Incorporate relevant icons and high - quality images to enhance the visual appeal and provide visual cues to users. Icons should be simple and recognizable, and images should support the content and user tasks.

5.2 Define the information architecture

Organize the information in a logical order, with a clear introduction, main body and conclusion. Classify and structure the information in a way that is meaningful to users.

6. Project Deploy

6.1 Deployment Strategy

How should the front-end and back-end services be deployed — on the same server or separately?

- The back-end hosts a large language model (LLM), which is resource-intensive and requires GPU acceleration(GPU – 16 GB VRAM, 8+ TFLOPS SP performance)[4].
- The front-end is lightweight and mainly handles UI rendering and user interaction(2 vCPUs, 4 GiB memory).

Separating the two avoids performance interference and improves scalability and maintainability[5].

Reference:

1. **Tailwind CSS.** "A utility-first CSS framework for rapid UI development." [Online]. Available: https://tailwindcss.com/
2. **Hugging Face Open LLM Leaderboard.** "Instruction-Following Evaluation (IFEval) Rankings." [Online]. Available: https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard
3. **LangChain Documentation.** "LLM Orchestration Framework." [Online]. Available: https://docs.langchain.com/
4. **Tencent Cloud HAI (Hyper AI Infrastructure)**. "Tencent Cloud's GPU-accelerated platform for deploying large-scale AI models." [Online]. Available: https://cloud.tencent.com/product/hai
5. **Google Cloud Documentation.** "Best practices for deploying AI applications using GPU and Cloud Run services." [Online]. Available: https://cloud.google.com/run/docs/configuring/services/gpu-best-practices
6. **LangChain Blog.** "Using LLaMA 3 with LangChain: Native Integration Guide." [Online]. Available: https://python.langchain.com/docs/integrations/providers/

Supervisor assessment	Please, rate your team (1) effort, (2) project progress and (3) their self-reflection for milestone 1 Rating scale 1-10 as per standard marking scheme, ie 5 is a Pass and 7 is a credit. Add some comments to explain your rating

Effort: Progress: Reflection:	